

**Three possible
BiLSTM-based
approaches for
Event Detection**

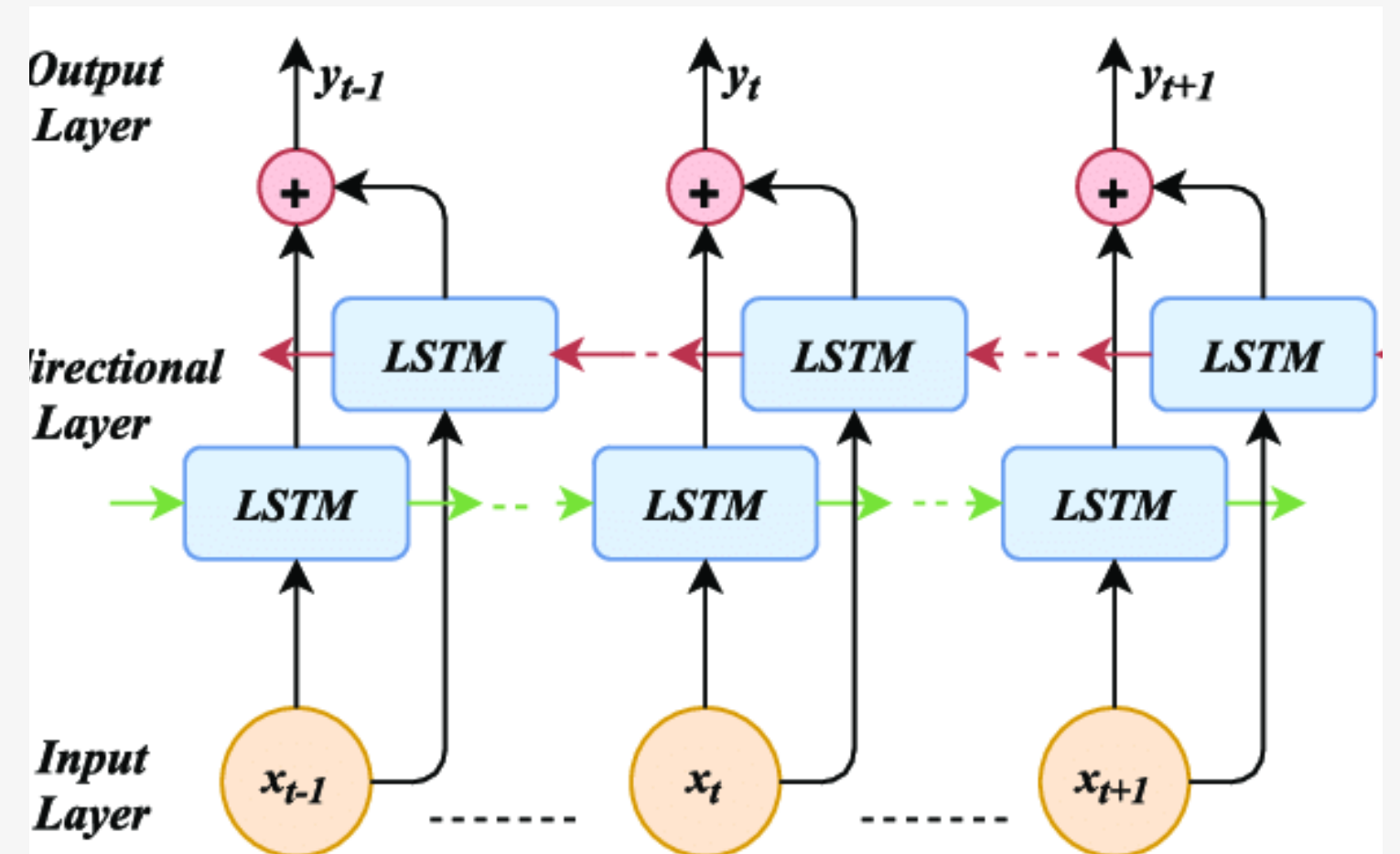
MNLP - Homework 1
Antonello Giorgio
1836529

What does my approaches involve?

Bidirectional Long-Short Term Memory (Bi-LSTM)-based architecture.

Moreover, addition of:

- Word embeddings
- Part-Of-Speech (POS) embeddings
- Conditional Random Fields (CRF)

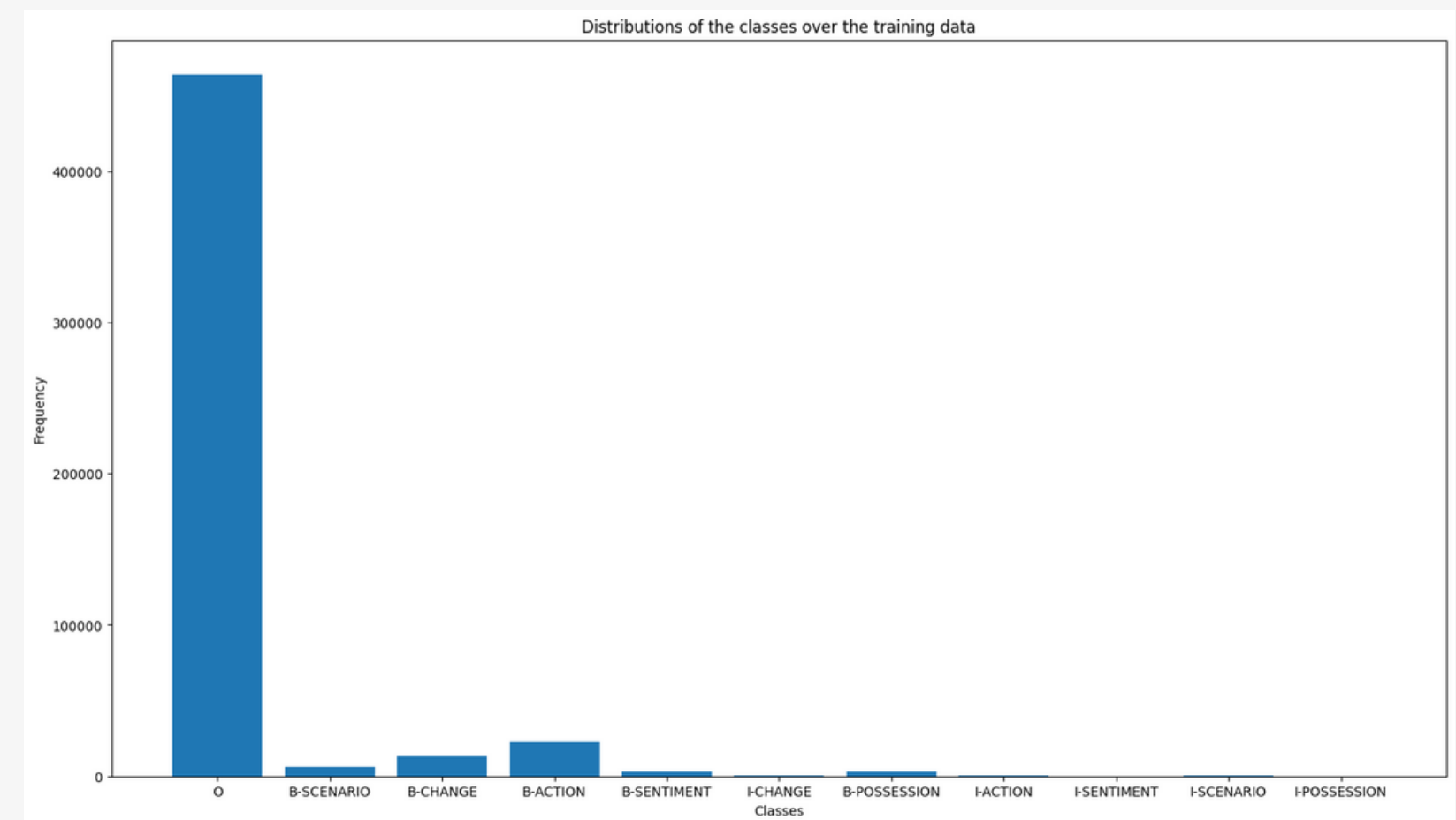


Data Preparation

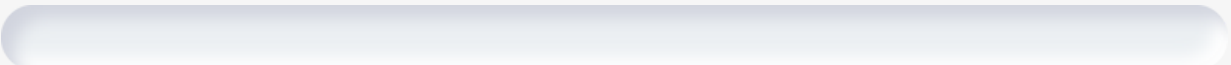
The Dataset

- Each sample is a sentence annotated with **event triggers** and their **classes**.
- event set = **SEN**:Sentiment, **CHA**:Change, **ACT**:Action, **SCEN**:Scenario, **POSS**:Possession
- **BIO** format to represent event trigger spans => **11 possible labels**
- **predominance** of 'O', "**B-ACTION**" and "**B-CHANGE**" classes

B-SENTIMENT	I-SENTIMENT
B-CHANGE	I-CHANGE
B-ACTION	I-ACTION
B-SCENARIO	I-SCENARIO
B-POSSESSION	I-POSSESSION
O	



Preprocessing



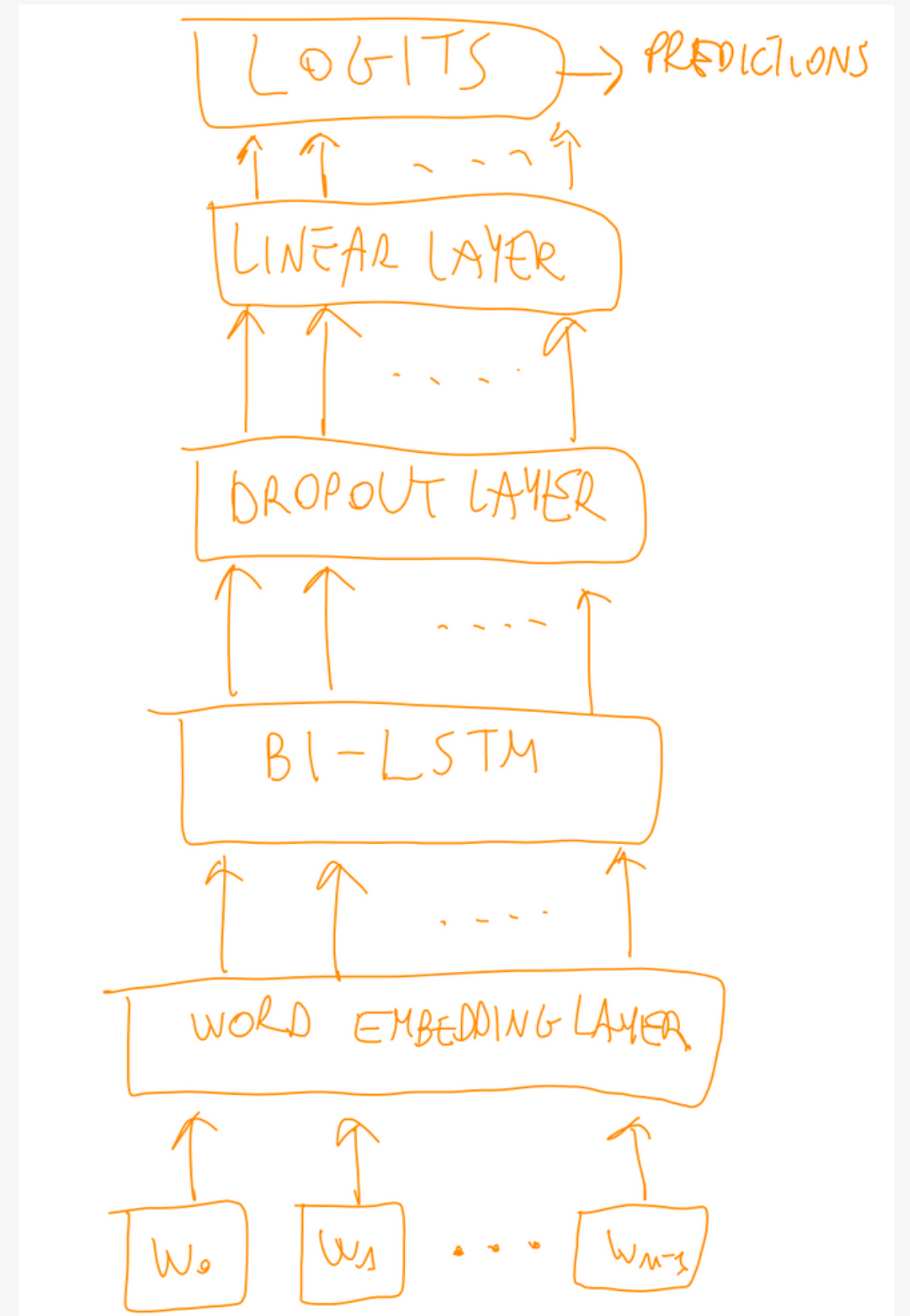
```
idx": 10,  
tokens": ["The", "meeting", "will", "take", "place", "today"],  
labels": ["0", "B-ACTION", "0", "B-SCENARIO", "I-SCENARIO", "0"]
```

- obtain POS tags from sentences through NLTK
- convert tokens, POS tags and event labels to numbers through vocabularies
- lowercase format vocabulary (invariant w.r.t. words with capital letters)

Model Architectures

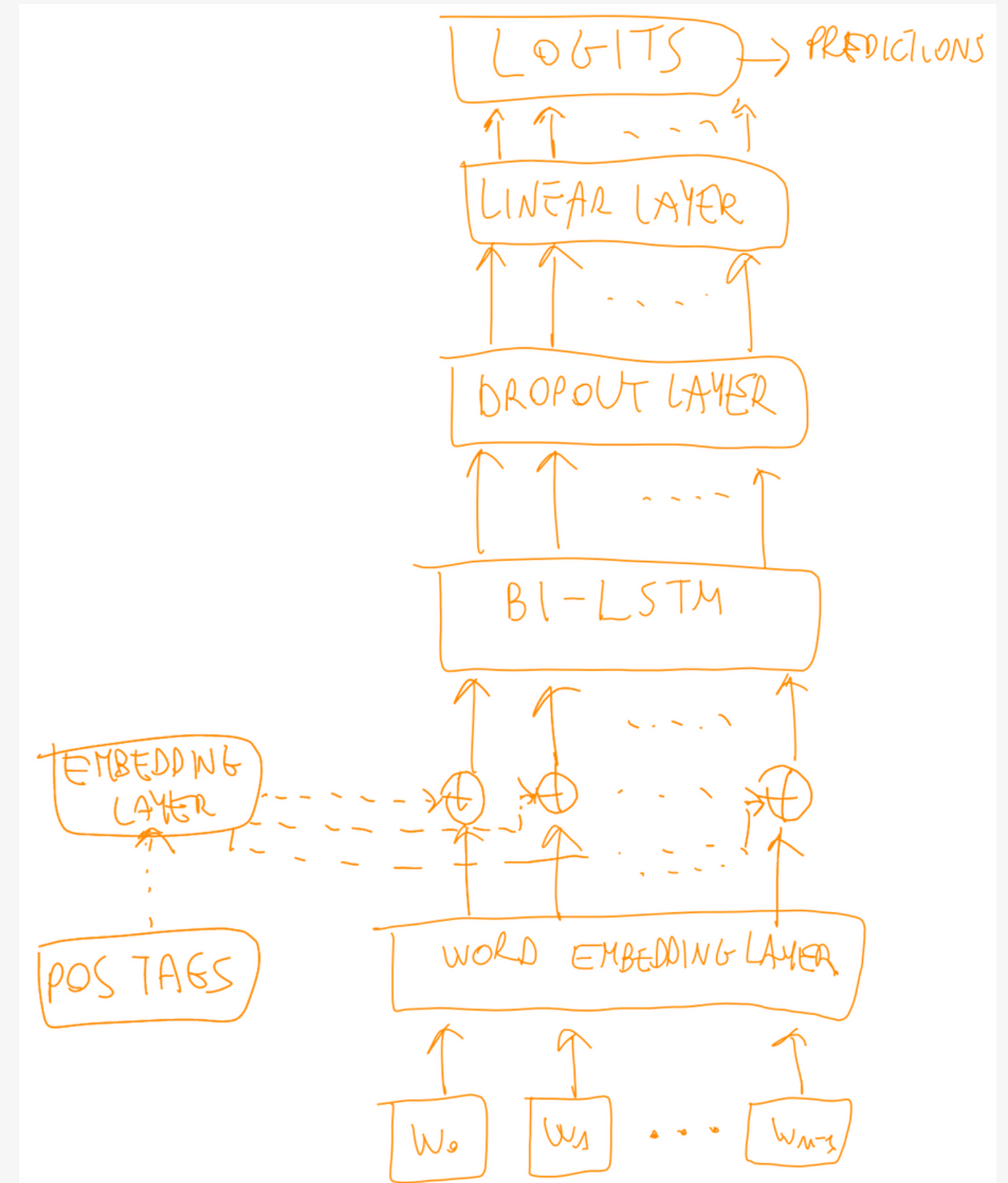
1st Approach: only Word Embeddings

- Retrieve them from *gensim* models (GloVe, etc.)
- **Bi-LSTM layer** takes the **word embeddings** of the sample sentence as **input** and processes them sequentially
- **output** of the Bi-LSTM **passed to a dropout layer**
- finally, **fully-connected linear layer** that produces a **prediction for each word**



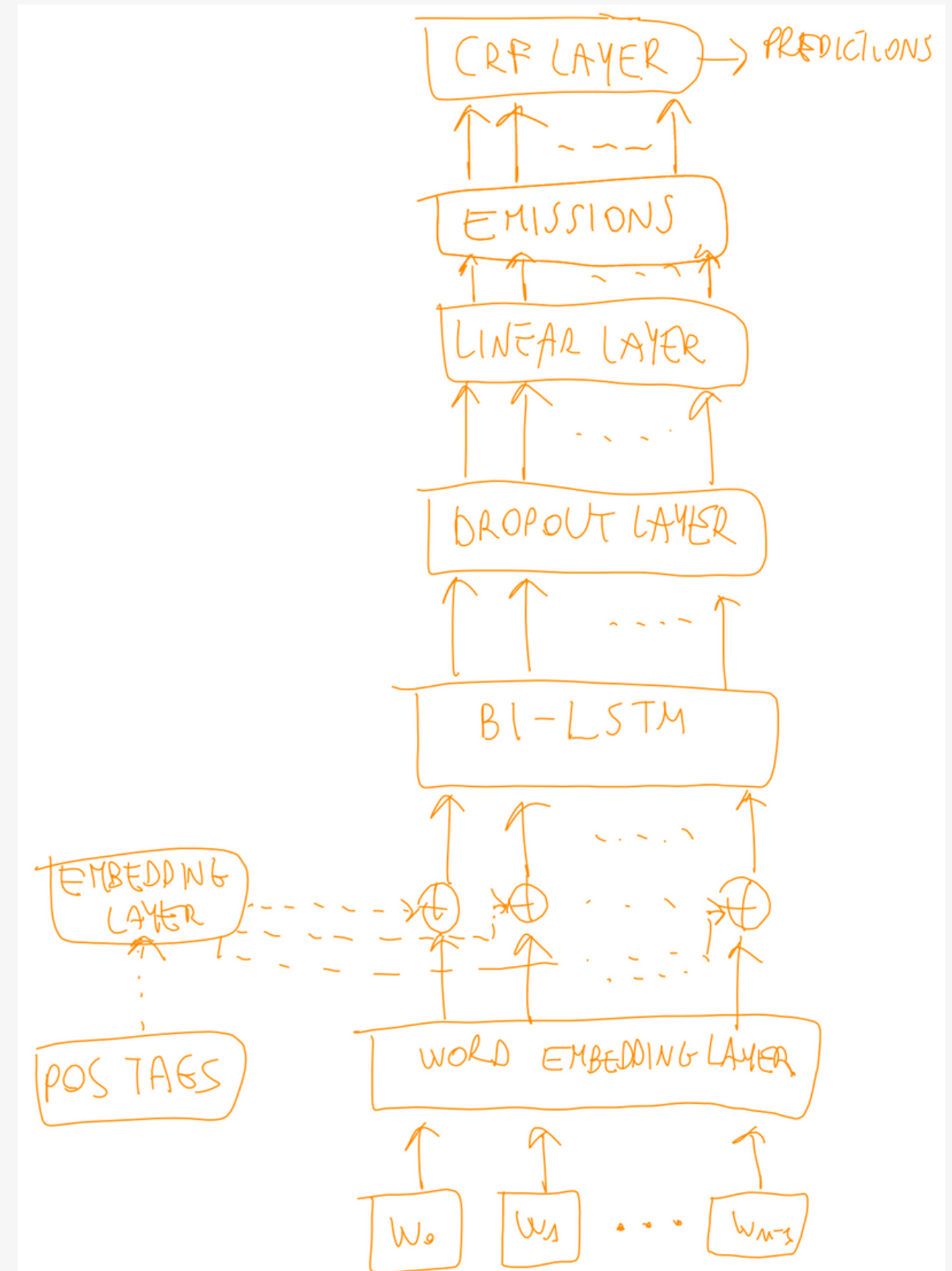
2nd Approach: Word + POS Embeddings

- **POS embeddings** identify a word's **syntactic function** in a phrase, such as whether it is a noun, verb, adjective, or adverb.
- **Bi-LSTM** now takes a **concatenation of NLTK-POS embeddings with gensim-word embeddings** as input.



3rd Approach: Word+POS Embeddings + CRF Layer

- pass output of Bi-LSTM layer through a **linear layer** to generate a sequence of **emissions** = **likelihood of each word** being part of an event.
- pass **emissions** through a **Conditional Random Field (CRF) layer** => models the **dependencies between adjacent words**.
- it computes the **negative log-likelihood loss between predicted emissions and true labels**, and returns the **final predictions** for the input sequence

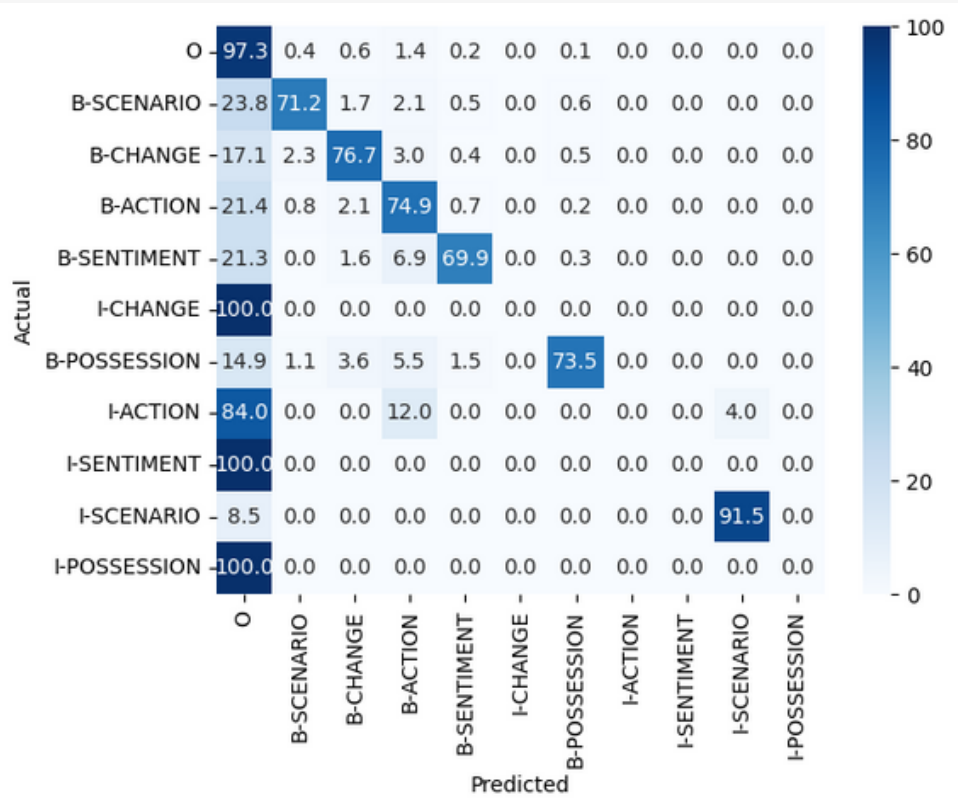


Experiments

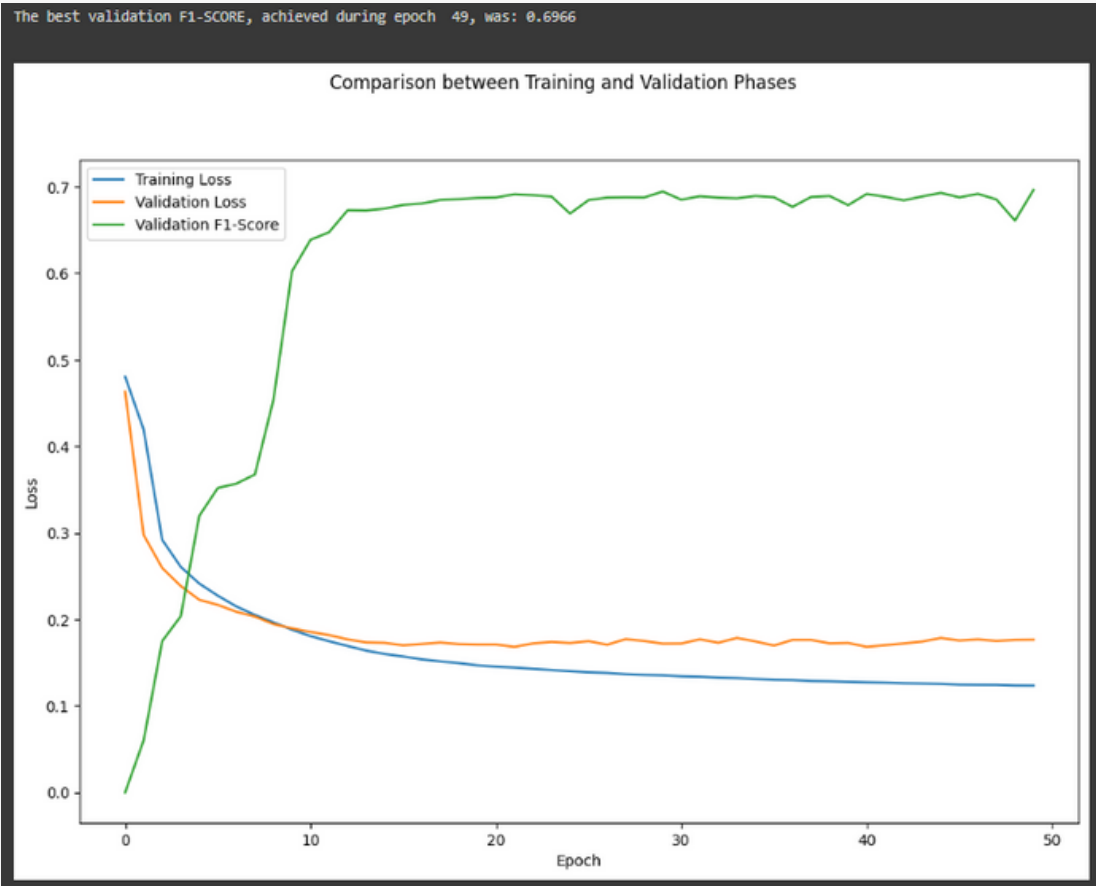
Word Embeddings

Hyperparameters	Values
Epochs	50
Batch size	32
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	5
Dropout	0.3
Lowercase tokens	False

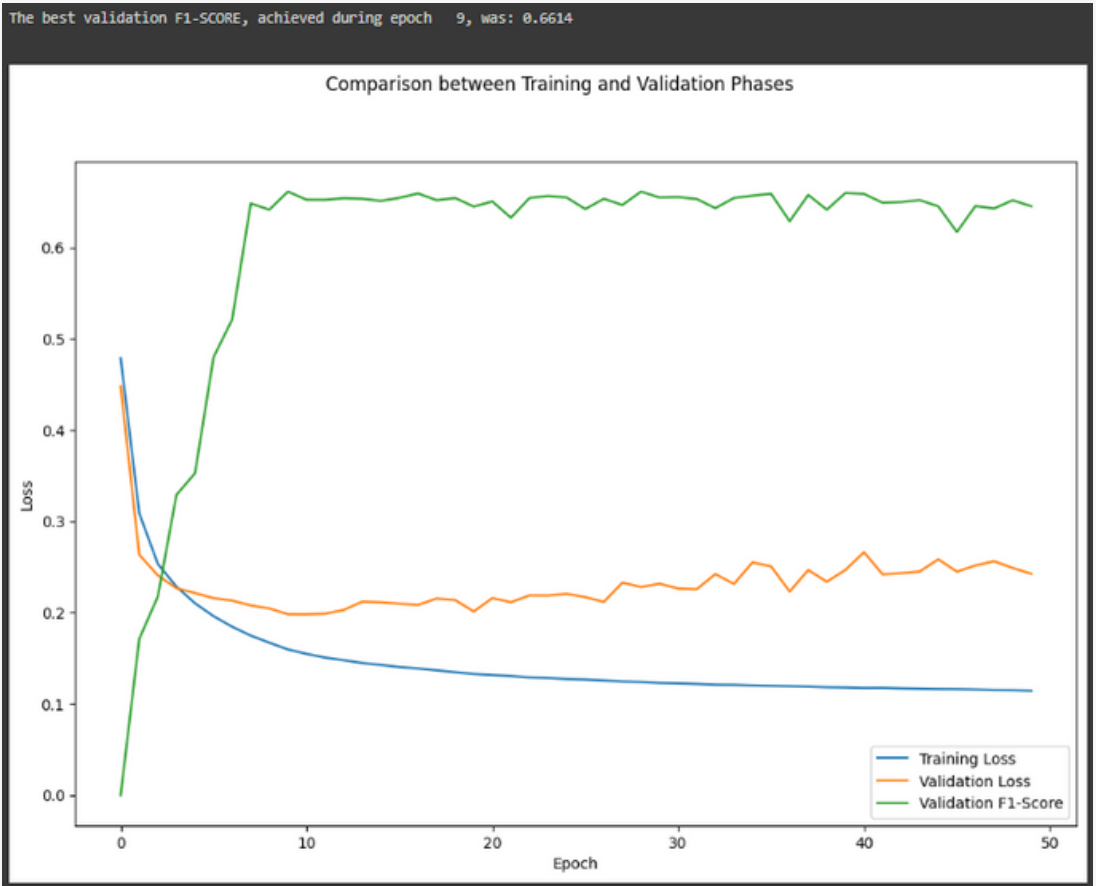
Test F1-Score
(GloVe 50) = 0.7040



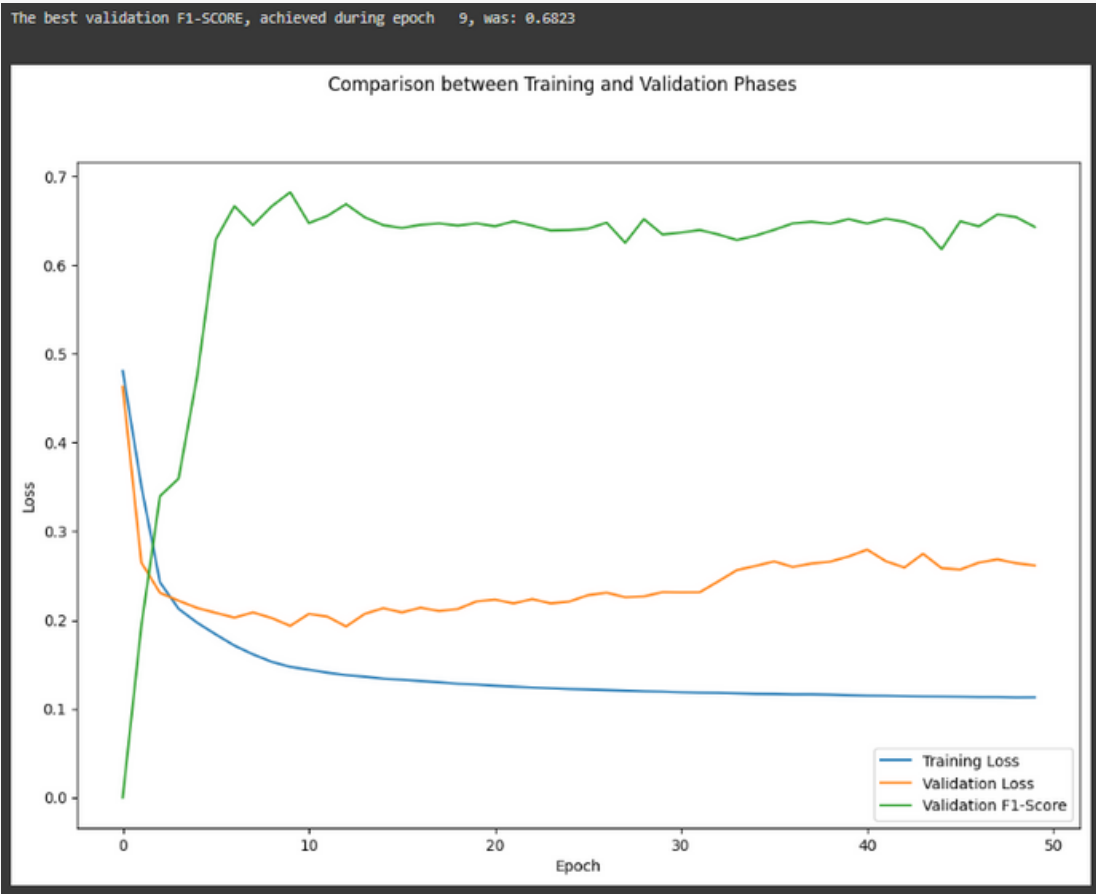
GloVe 50



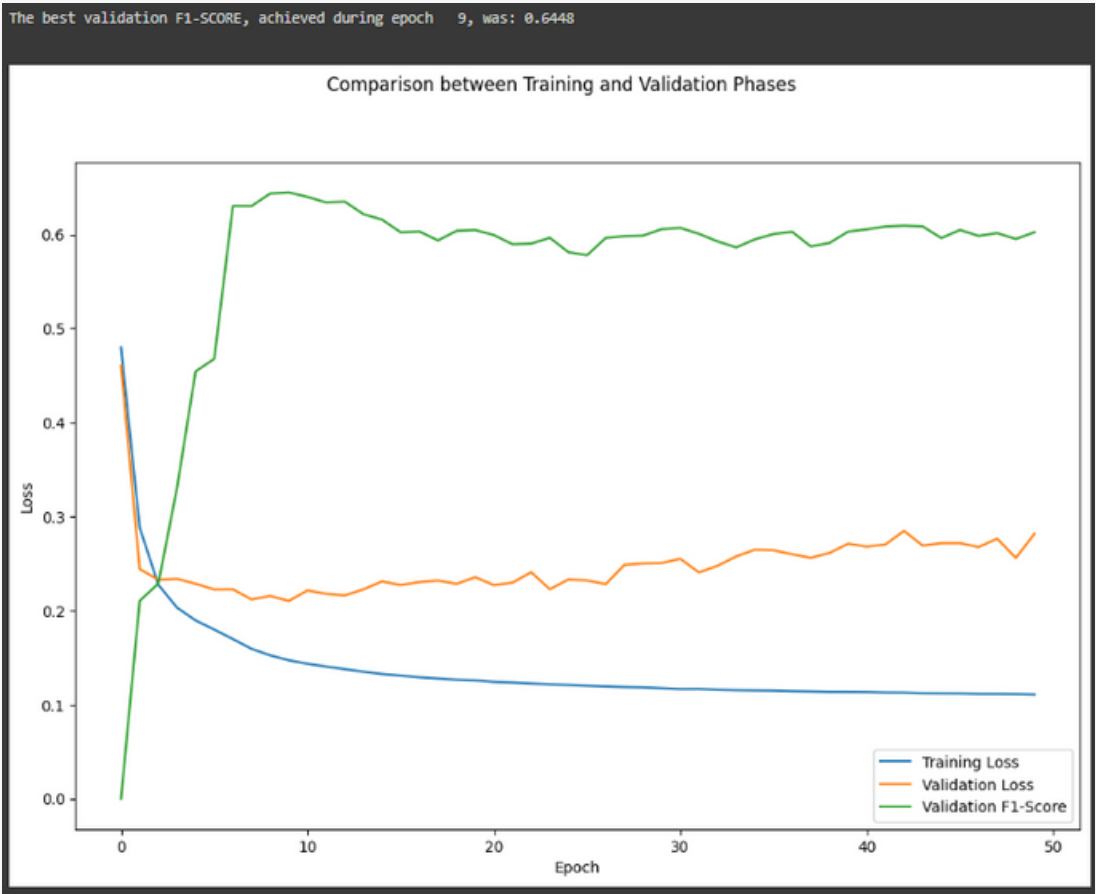
GloVe 100



GloVe 200



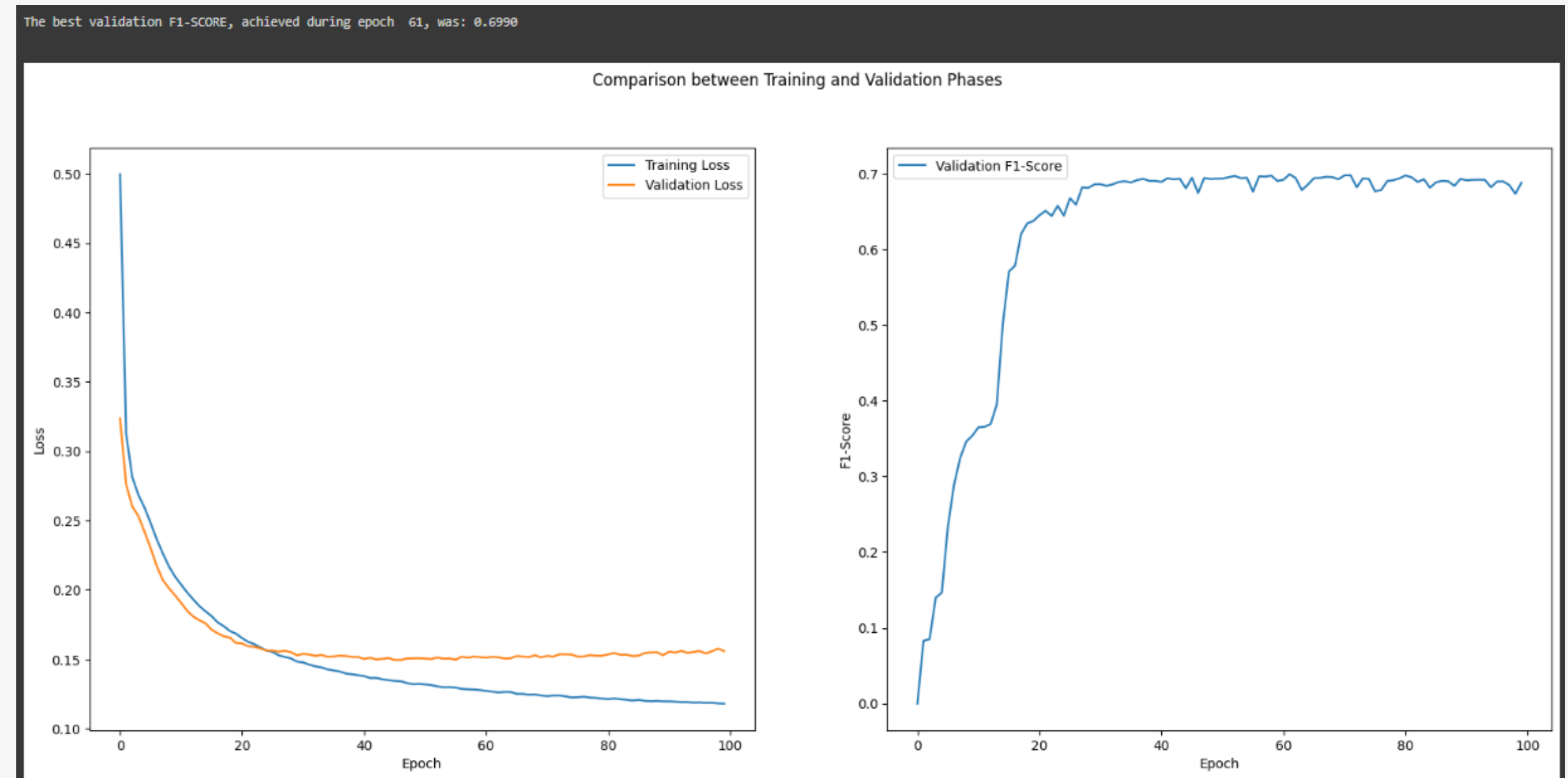
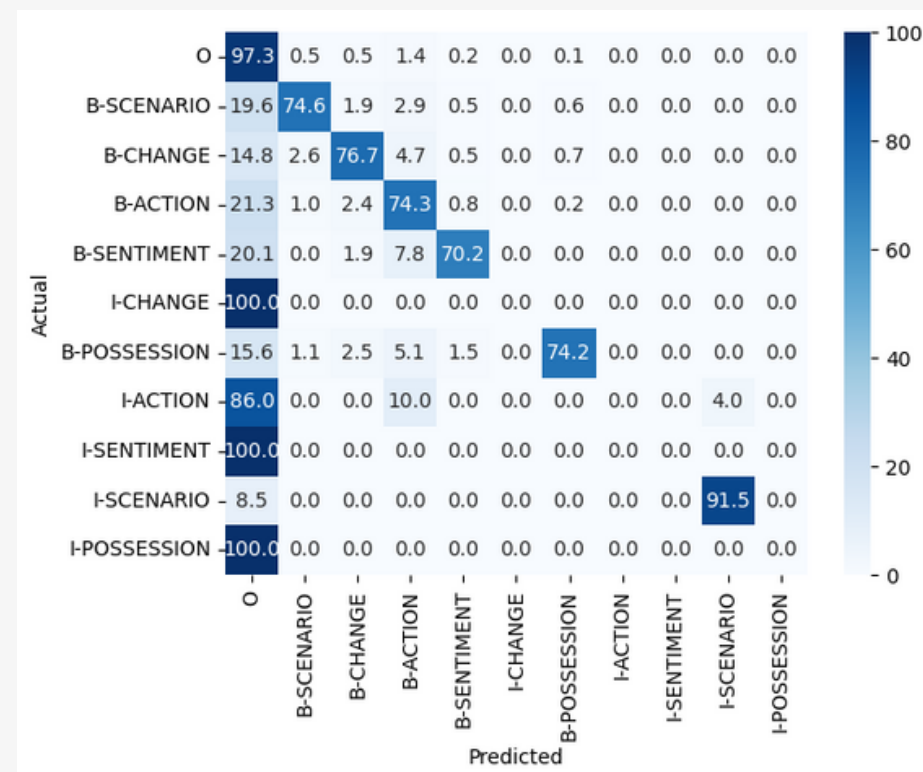
GloVe 300



Word+POS Embeddings - Experiment 1

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	True
POS embeddings dimension	50

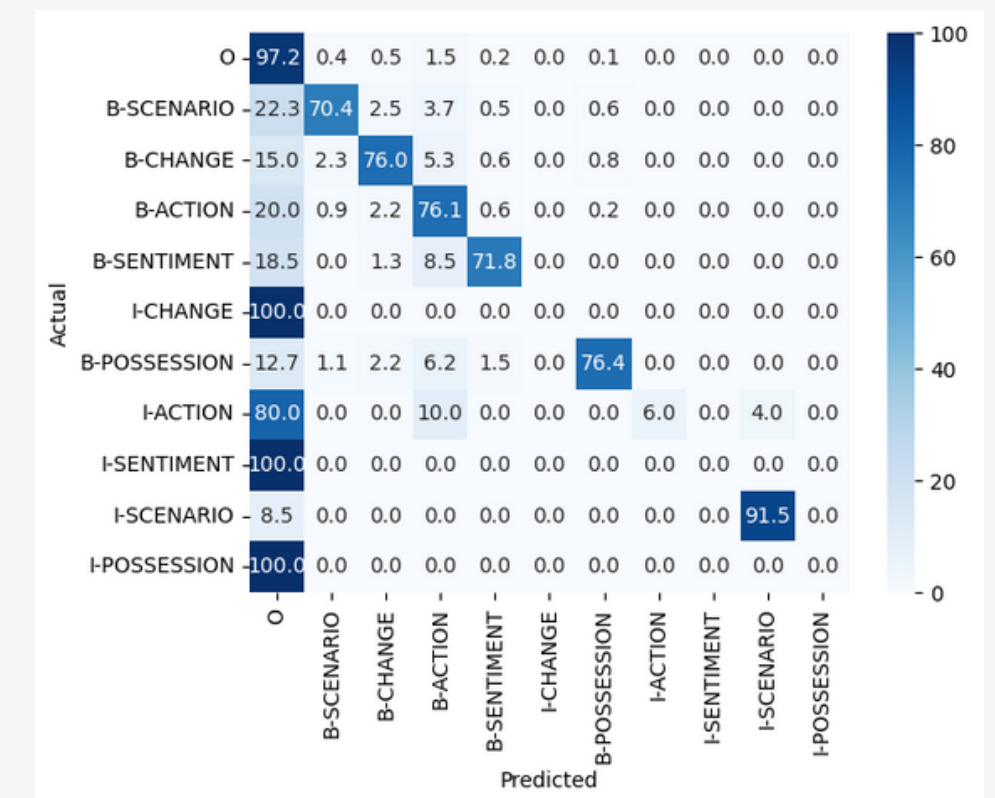
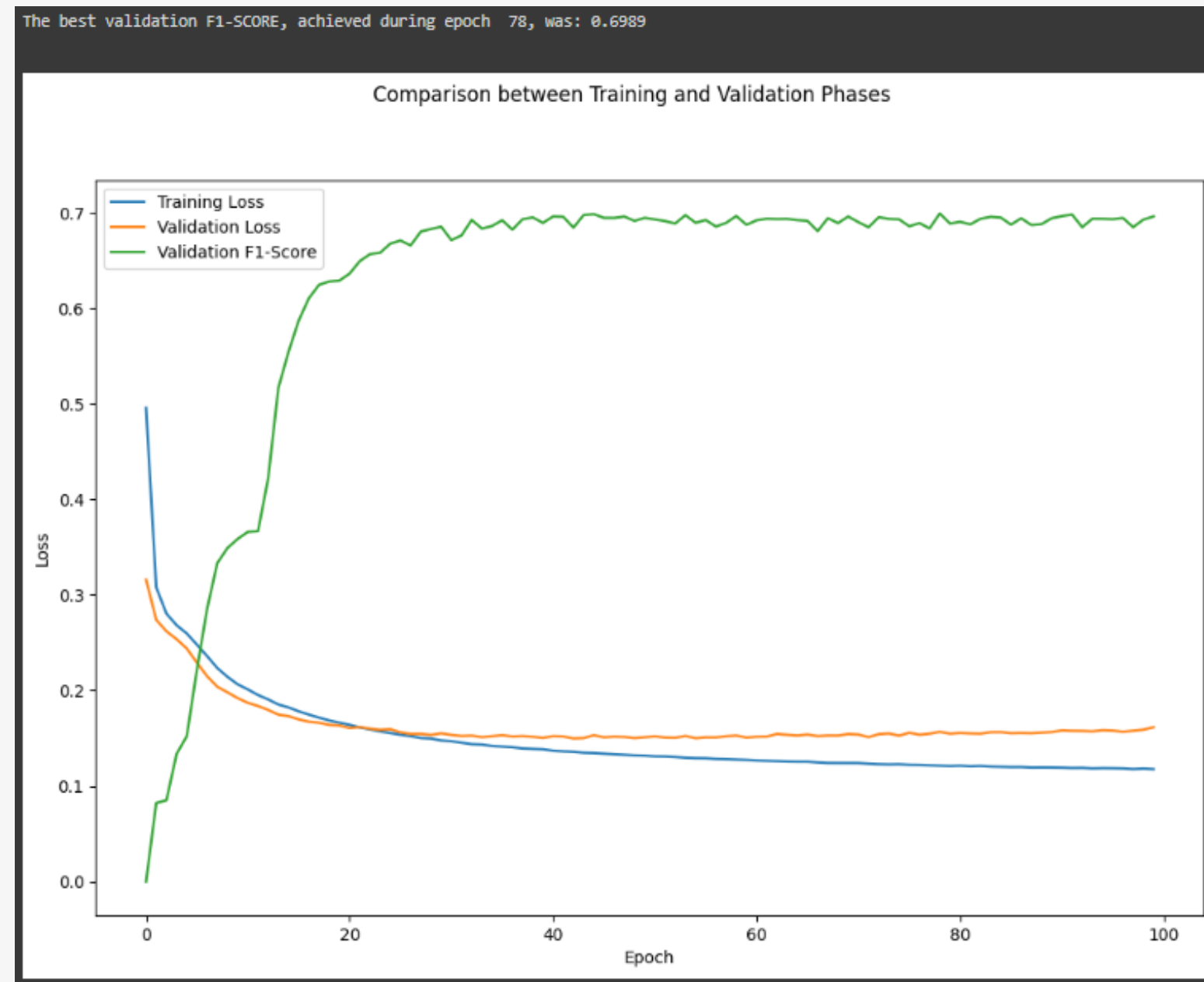
Test F1-Score =
0.7072



Word+POS Embeddings - Experiment 2

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	False
POS embeddings dimension	50

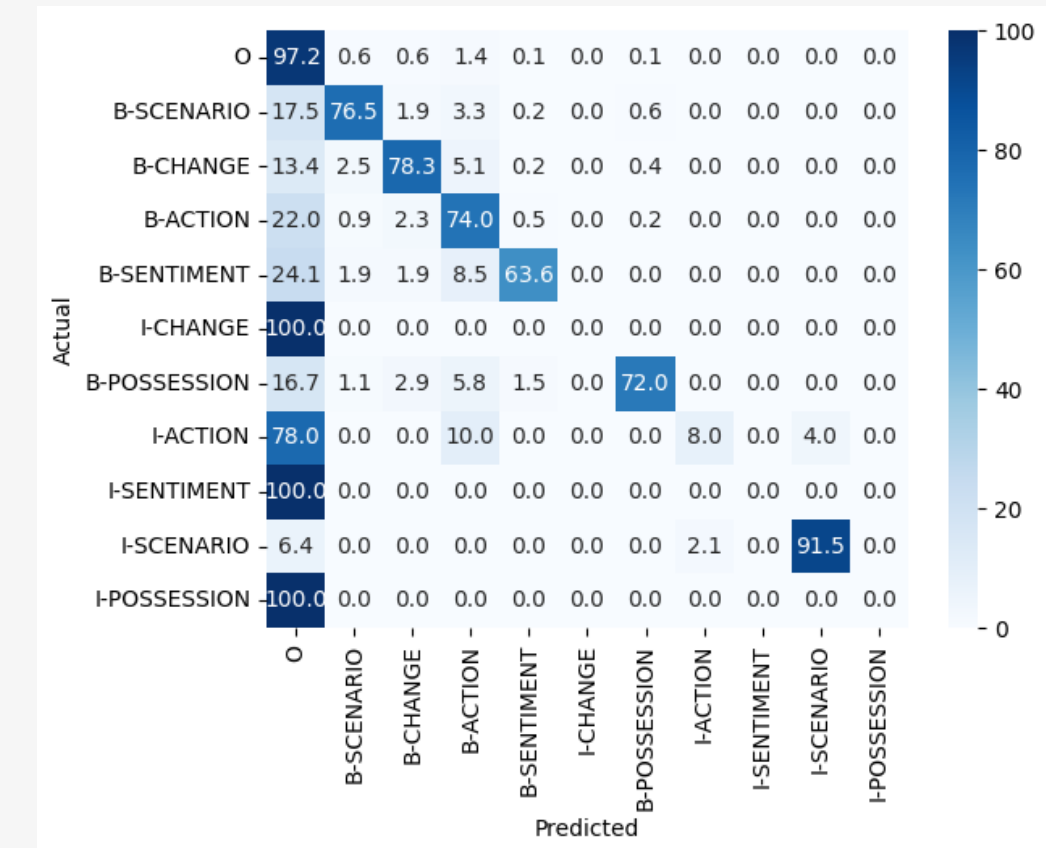
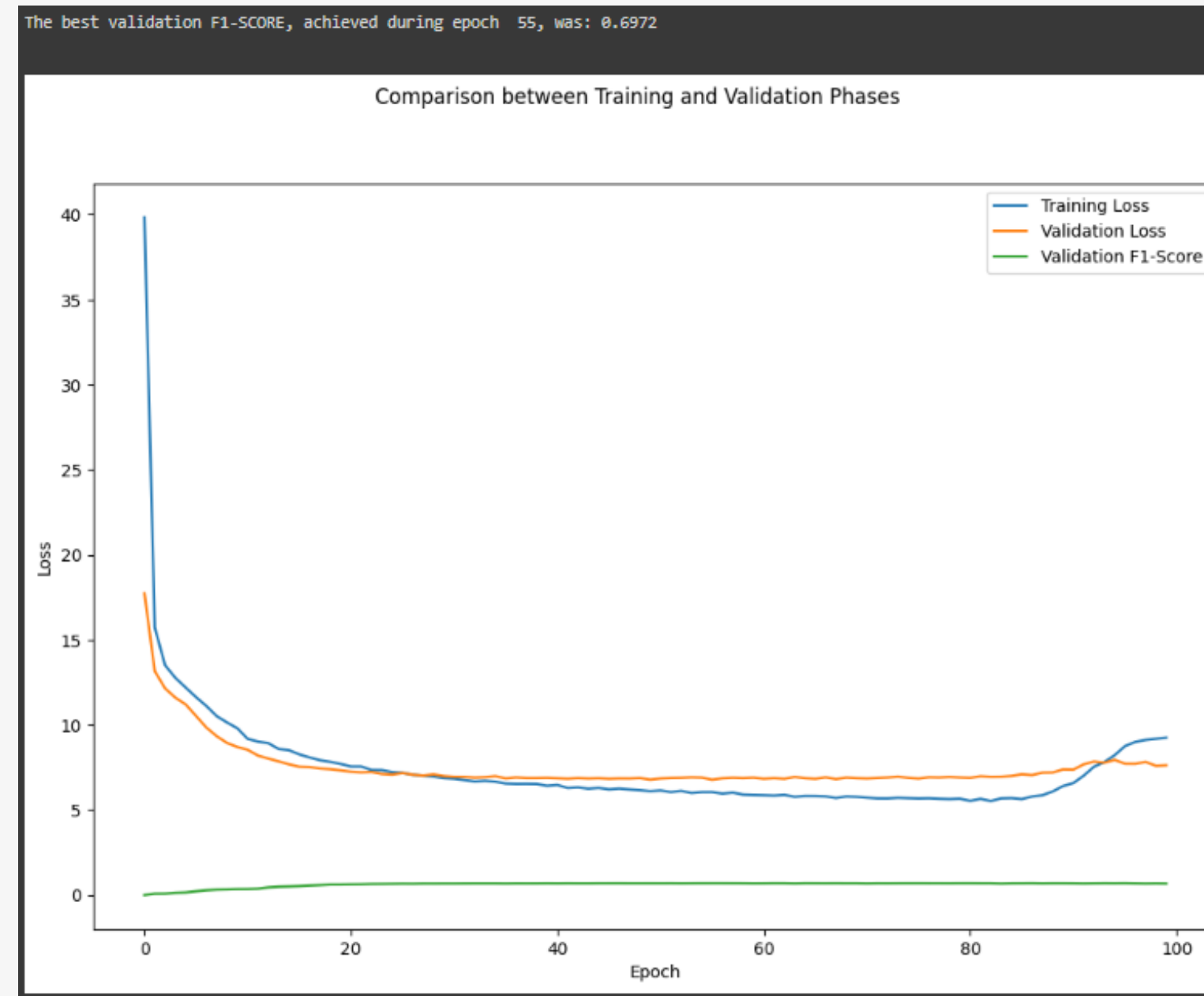
Test F1-Score = 0.7085



Word+POS Embeddings + CRF - Experiment 1

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	False
POS embeddings dimension	50

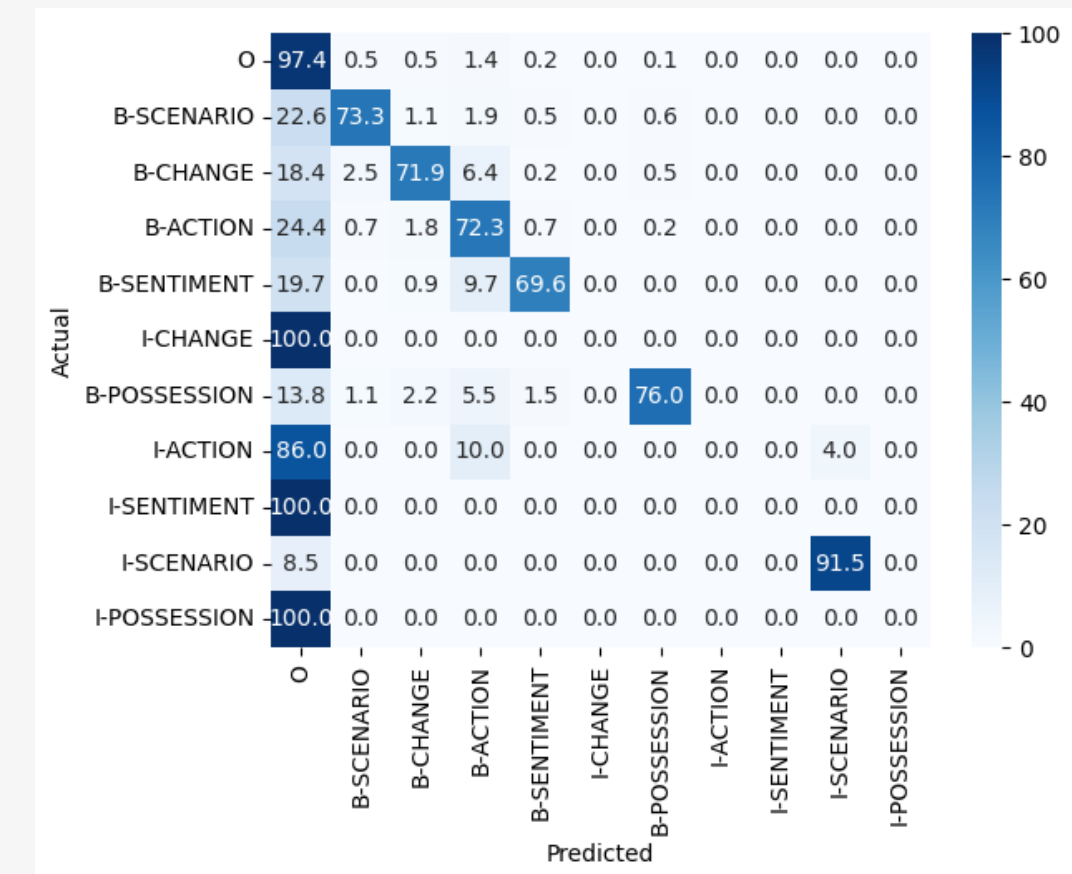
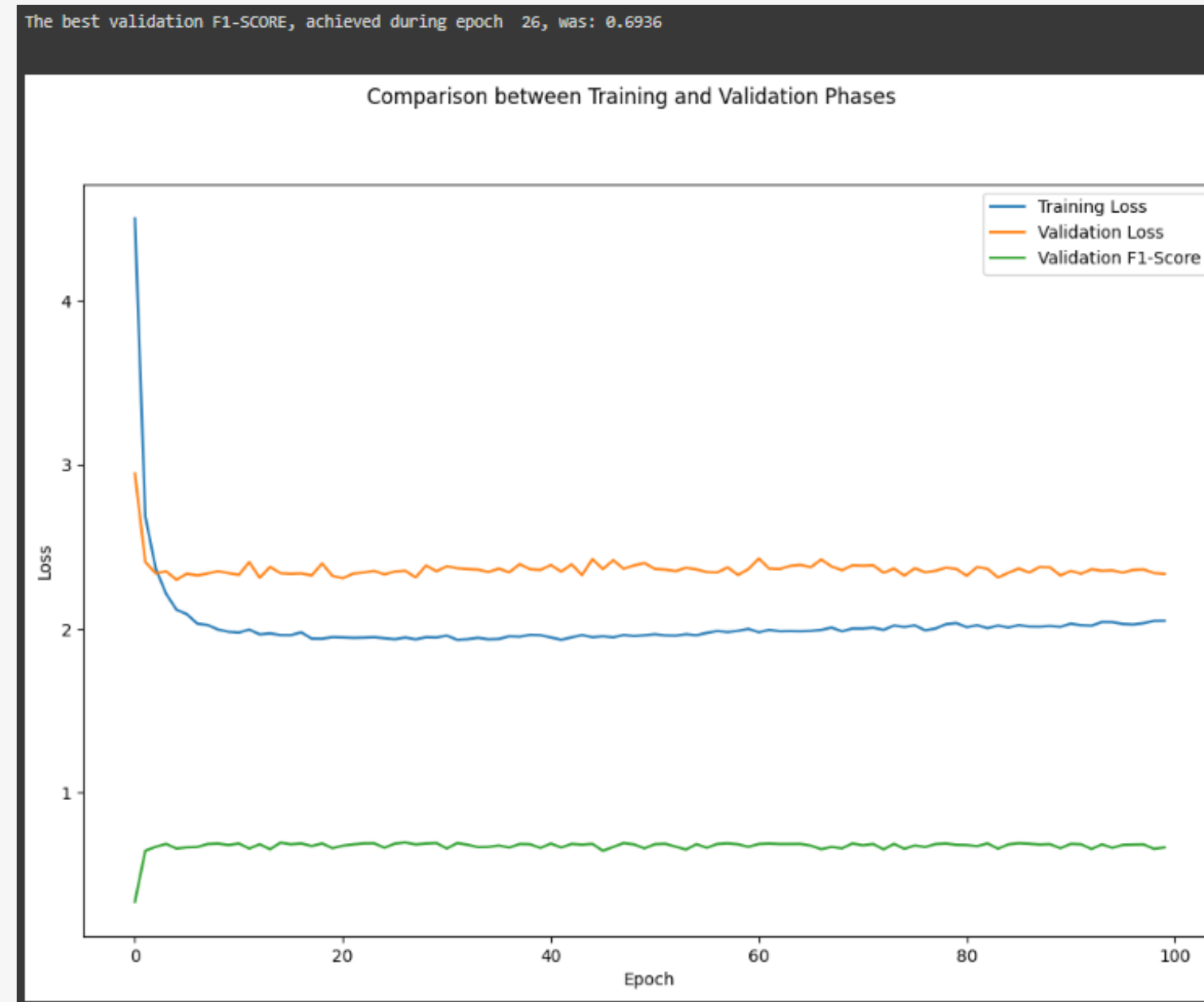
Test F1-Score = 0.7046



Word+POS Embeddings + CRF - Experiment 2

Hyperparameters	Values
Epochs	100
Batch size	32
Learning rate	0.01
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.4
Lowercase tokens	False
POS embeddings dimension	50

Test F1-Score = 0.7045



Conclusion



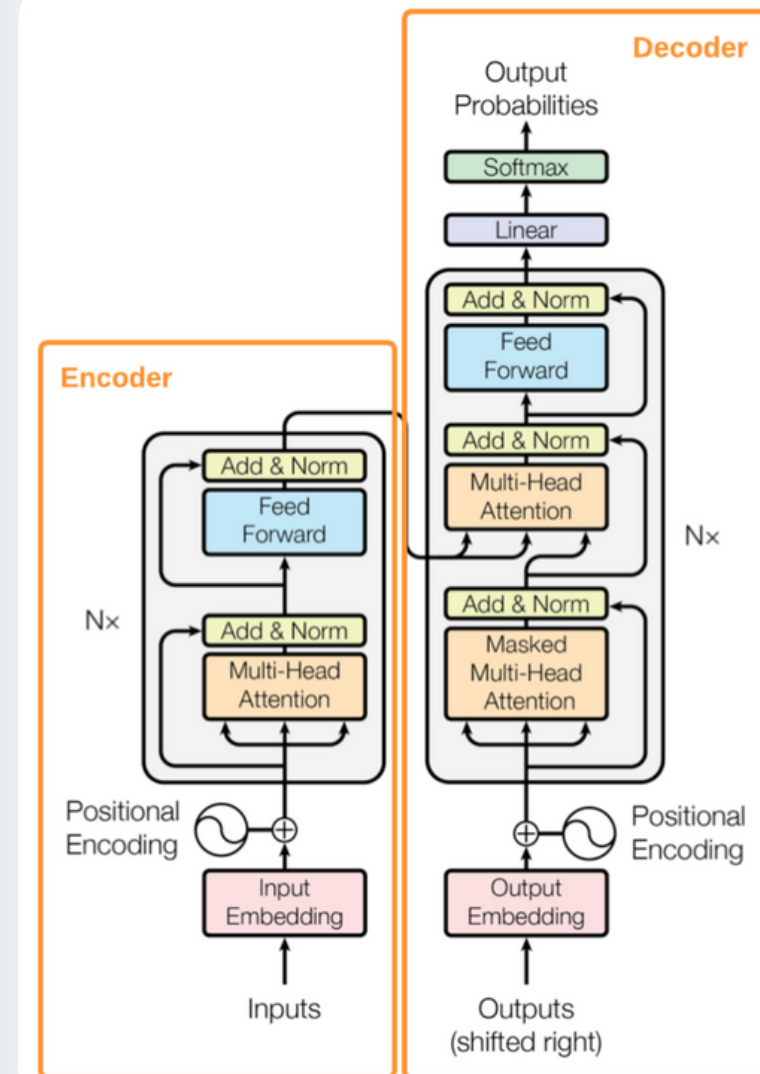
- **Results** showed that, even with **no significant changes** in terms of **F1-Score** metric, the **best performance** is associated with **medium model complexity**
- **word** embeddings **concatenated** with **POS** embeddings **improved the performance**
- **CRFs didn't** unexpectedly **help** detecting event-related words better

**POS-itive
Disambiguation:
WSD and the Power of
Transformers-based
Models combined with
POS Taggings**

**MNLP - Homework 2
Antonello Giorgio
1836529**

What does my approach involve?

Transformer-based architecture, in particular only the “**Encoder**” part. Moreover, focus on **Part-Of-Speech (POS)** embeddings.



Why POS tags for WSD?

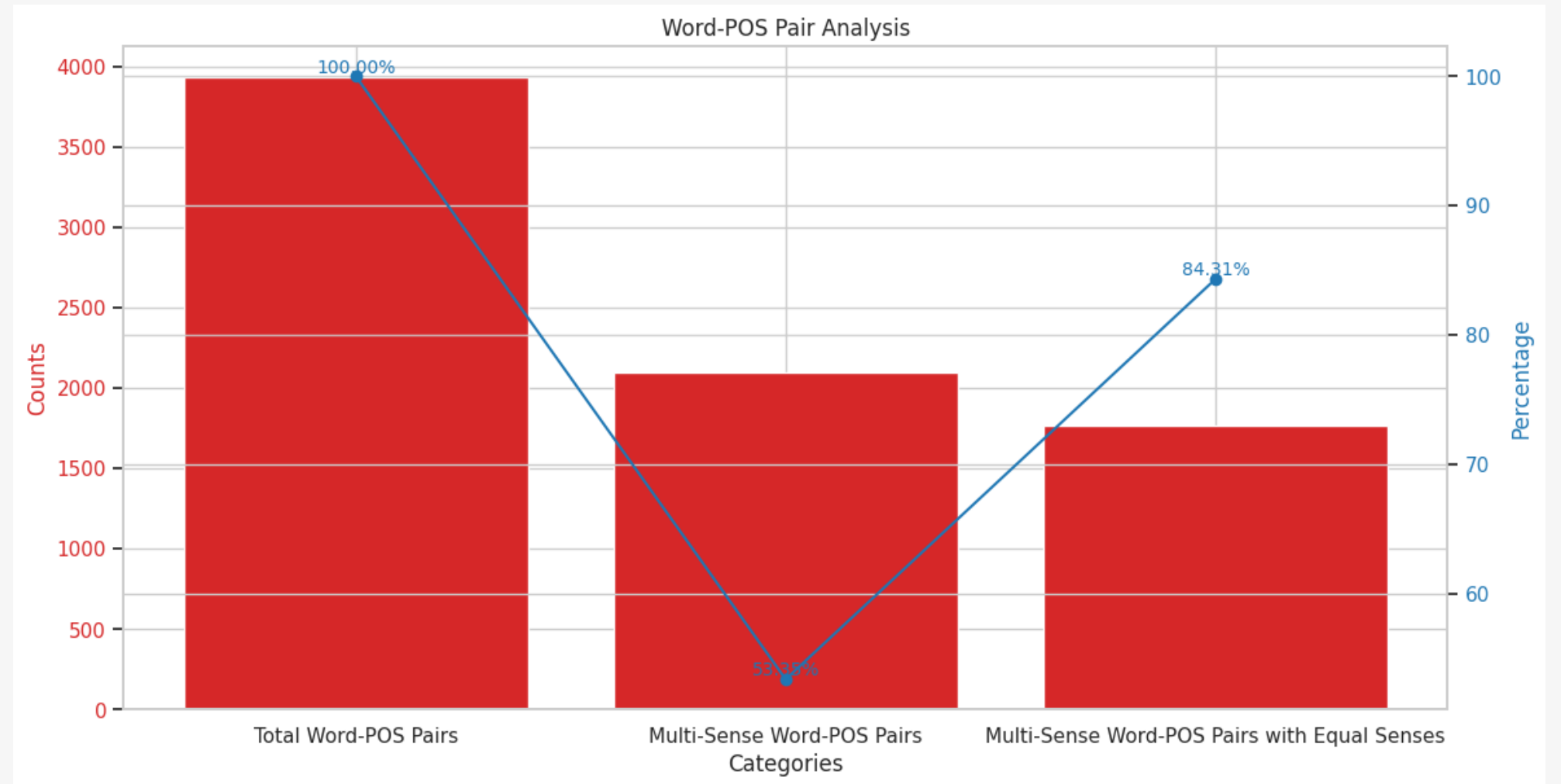
As previously said, they provide **information** about the **syntactic role** of a word in a sentence.

Idea:

word appears with same POS tag



most likely **same sense**



Data Preparation

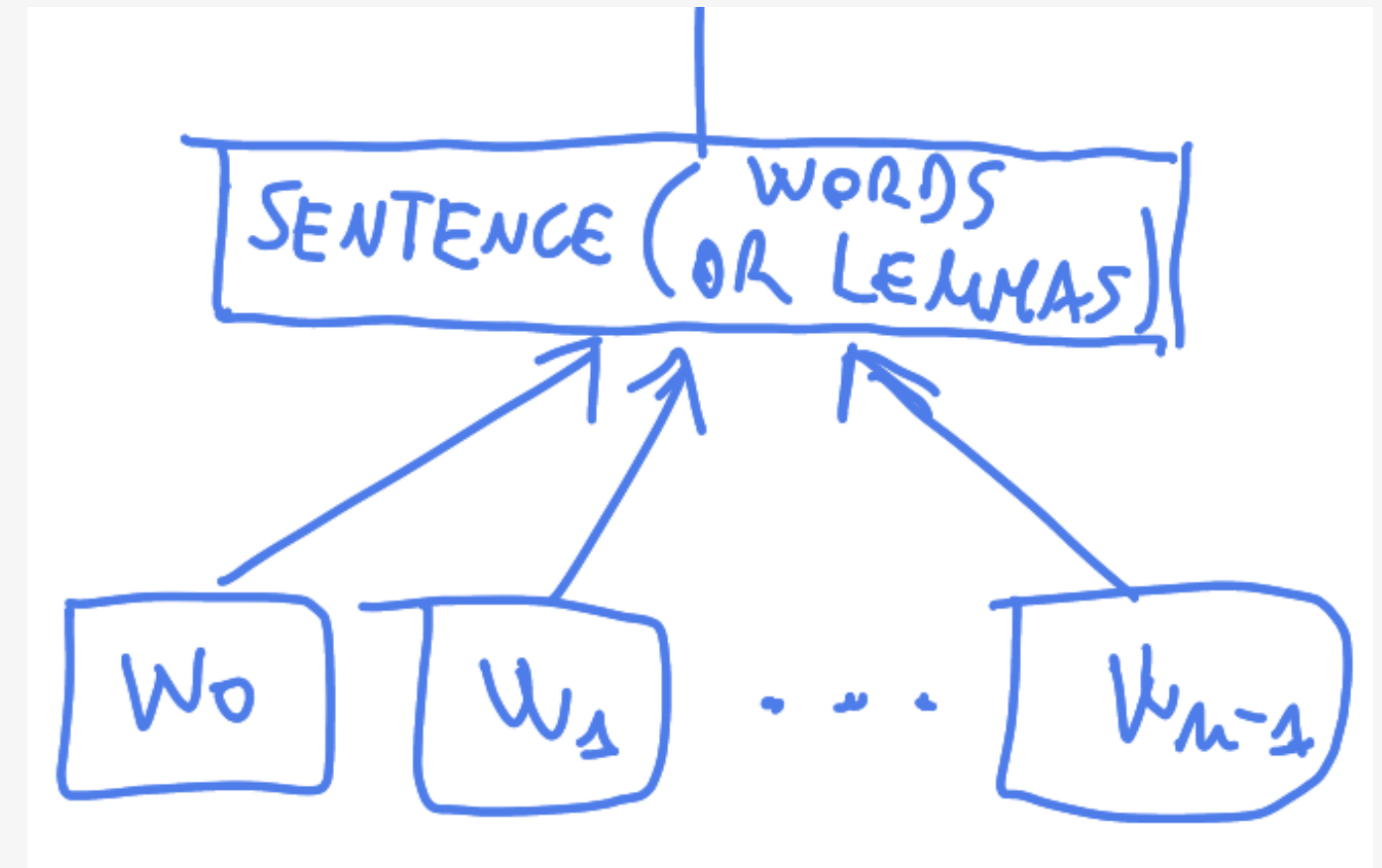
The Dataset

Coarse-grained dataset, containing **candidates** and correct **gold homonymy clusters** from **WordNet**.

```
"d000.s002": {
  "instance_ids": {
    "1": "d000.s002.t000"
    "5": "d000.s002.t001"
  }
  "lemmas": ["the", "race", "will", "take", "place", "today"]
  "words": ["The", "races", "will", "take", "place", "today"]
  "pos_tags": ["DT", "NOUN", "VB", "VB", "NOUN", "ADP"]
  "senses": {
    "1": "race.n.h.01"
    "5": "today.r.h.01"
  }
  "candidates": [
    "1": ["race.n.h.01", "race.n.h.02"]
    "5": ["today.r.h.01"]
  ]
}
```


Preprocessing

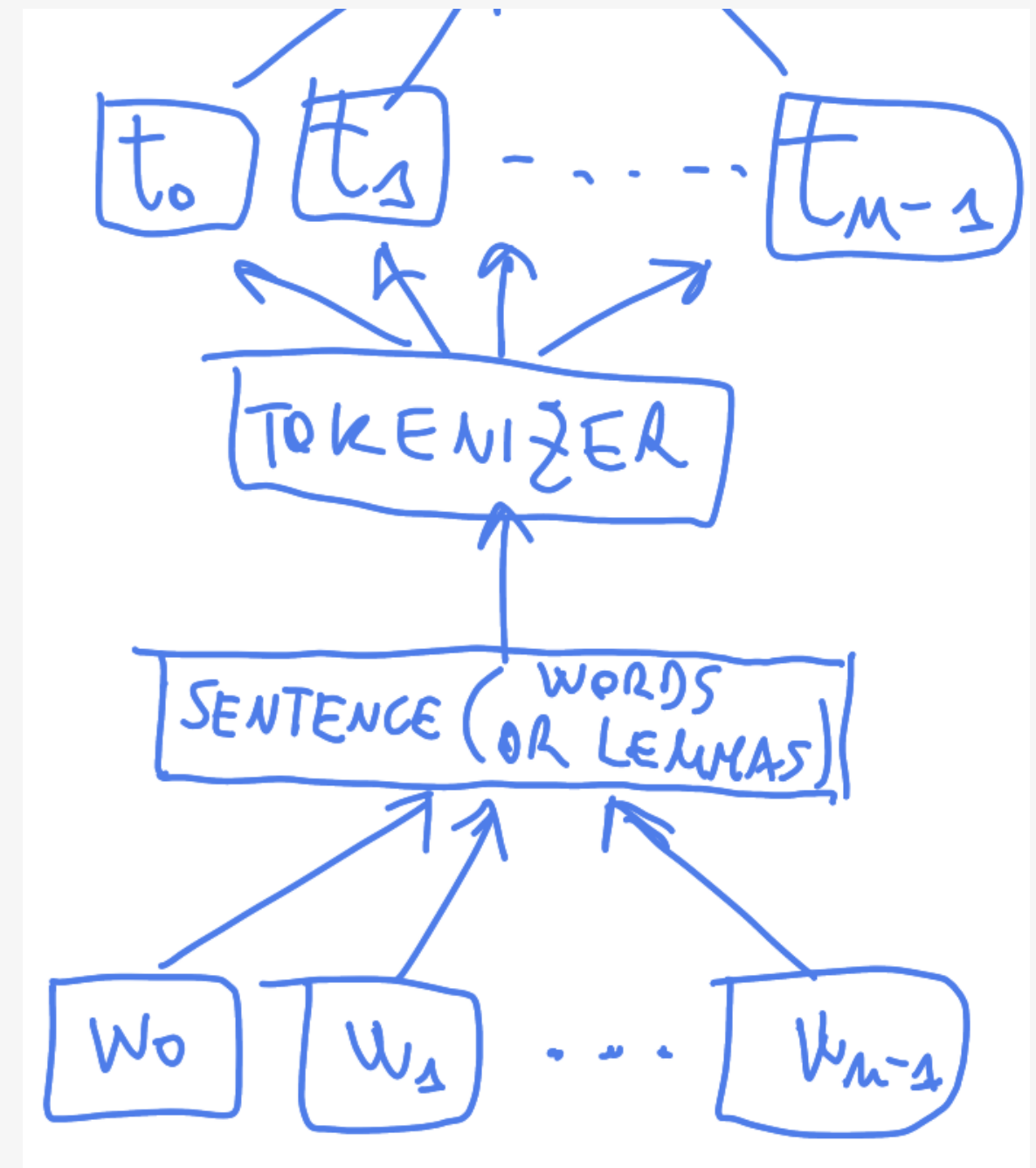
- obtain entire sentence from “**words**” or “**lemmas**” (boolean variable)
- convert “**pos_tags**” and “**senses**” to numbers through vocabularies
- use “**candidates**” to build senses vocabulary masks



Model Architecture and Features

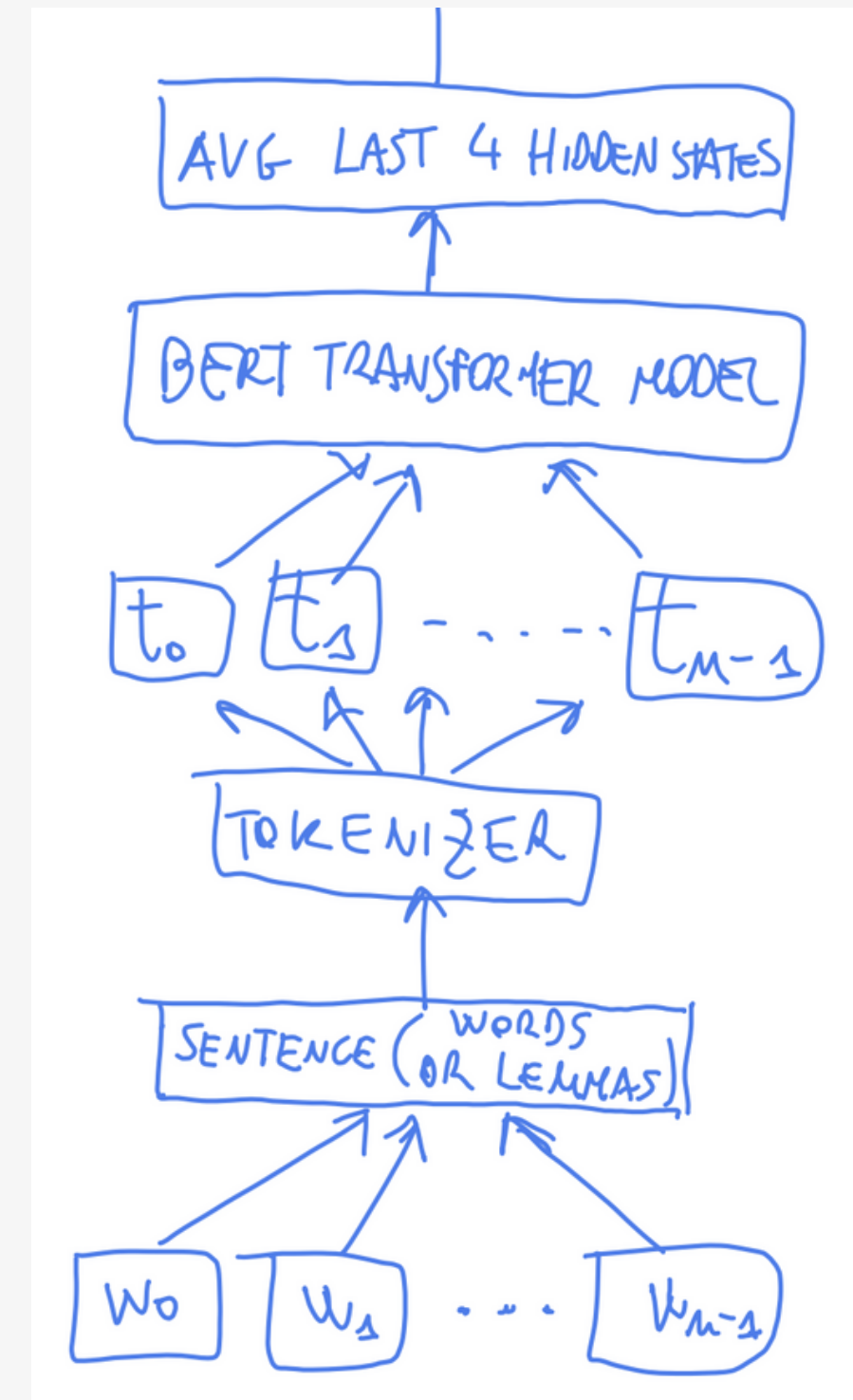
Tokenization

- **initial tokenization** of the **target word**, to understand **how it is subdivided** by the tokenizer
- **tokenize** whole **sentence**
- obtain **indices** related to the **positions** of the **tokens belonging to the target word**



Transformer Model

- **BERT** (Bidirectional Encoder Representations from Transformers) model and tokenizer
- **output** of the transformer = **average** of **last four hidden states**.
- **access target embeddings** through indices computed in tokenization process
- **average** target embeddings to obtain the **final target embedding**



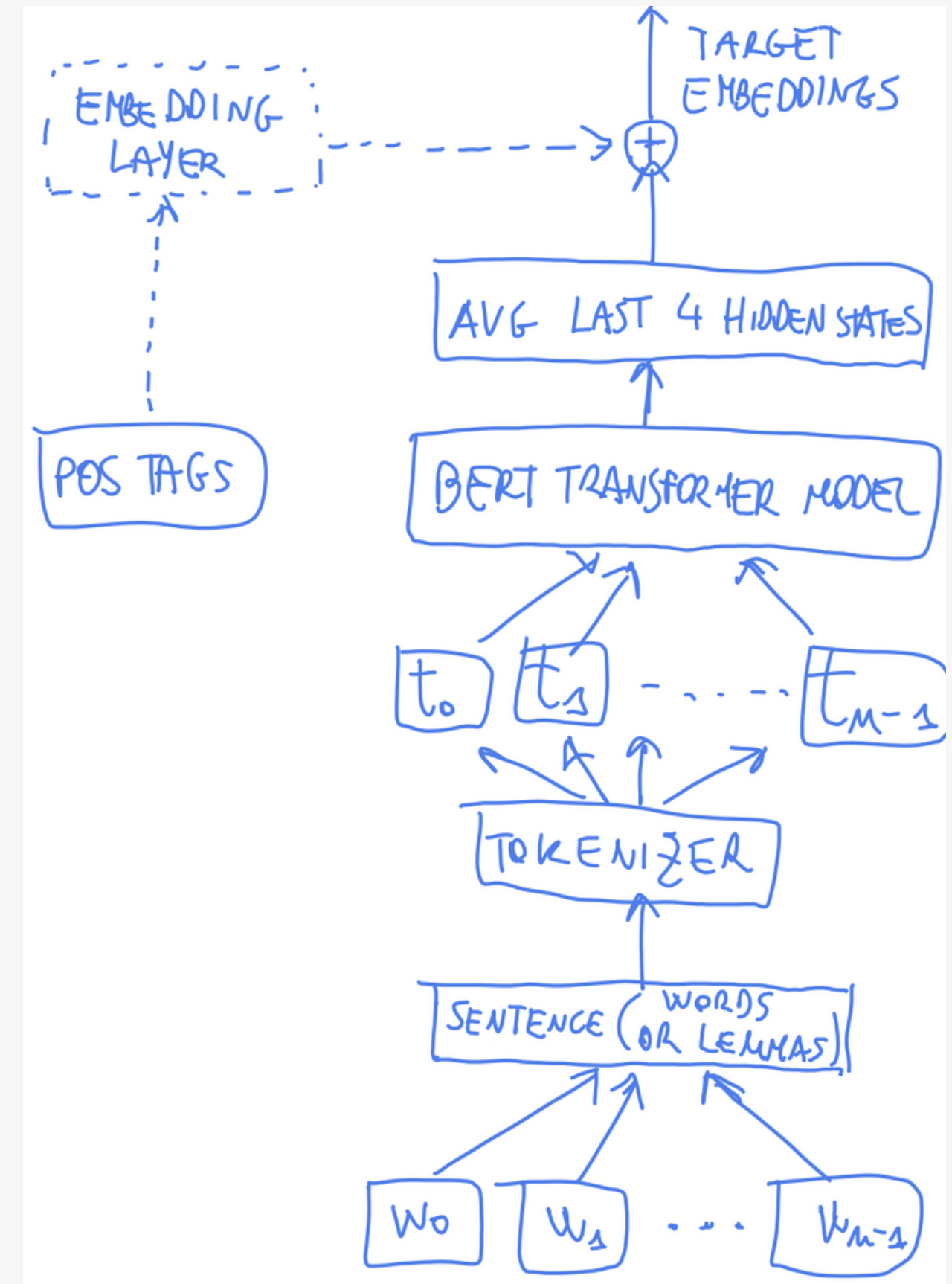
	Transformer Layers	Hidden Size	Attention Heads	Parameters
BERTbase	12	768	12	110M

POS Embedding Layer

- pass POS tags as a further input to the transformer model
- **obtain POS embedding from POS tags and combine them with transformer output**

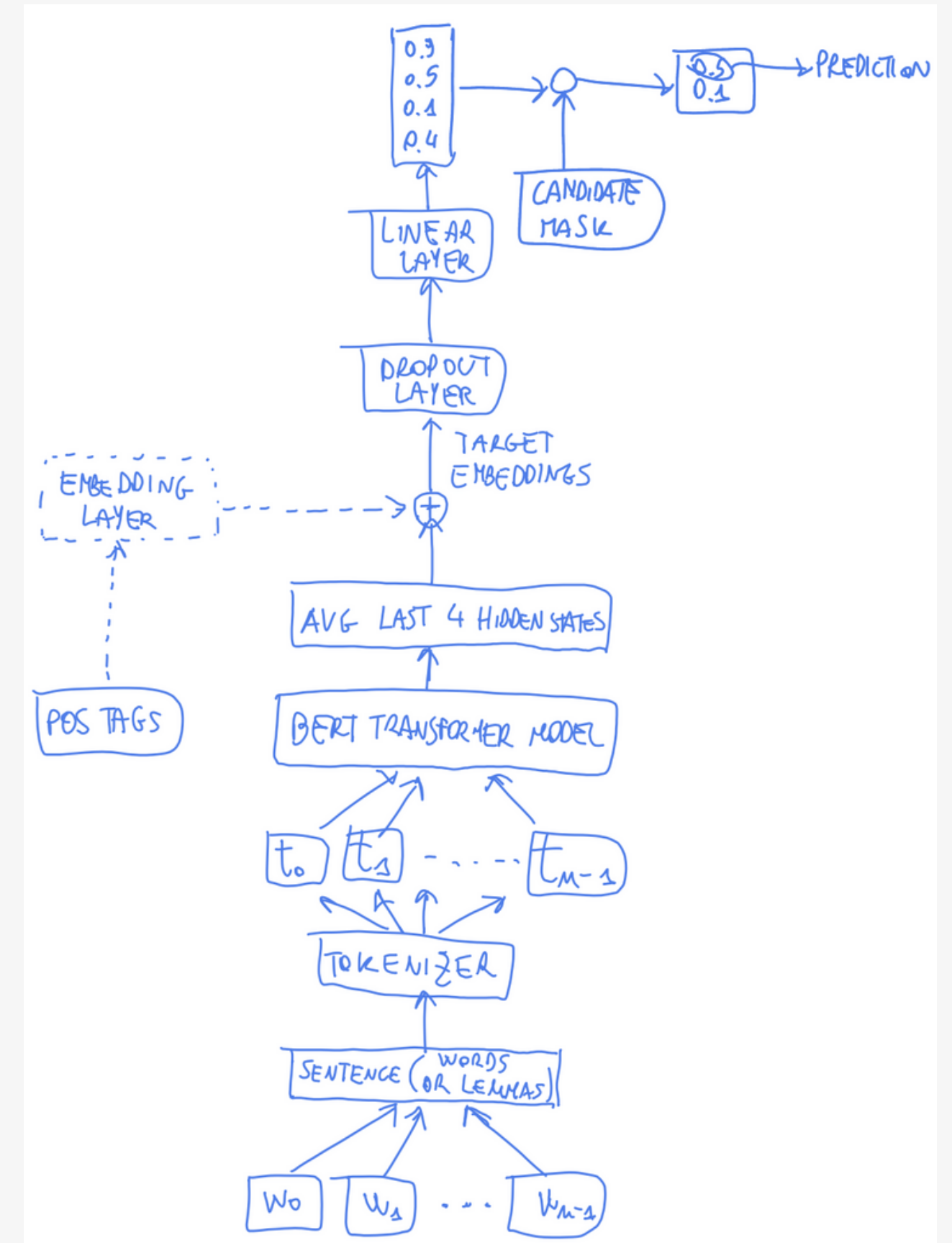
By choosing the second option:

- concatenate POS and Transformer embeddings.
- sum POS and Transformer embeddings.
- **average POS and Transformer embeddings.**



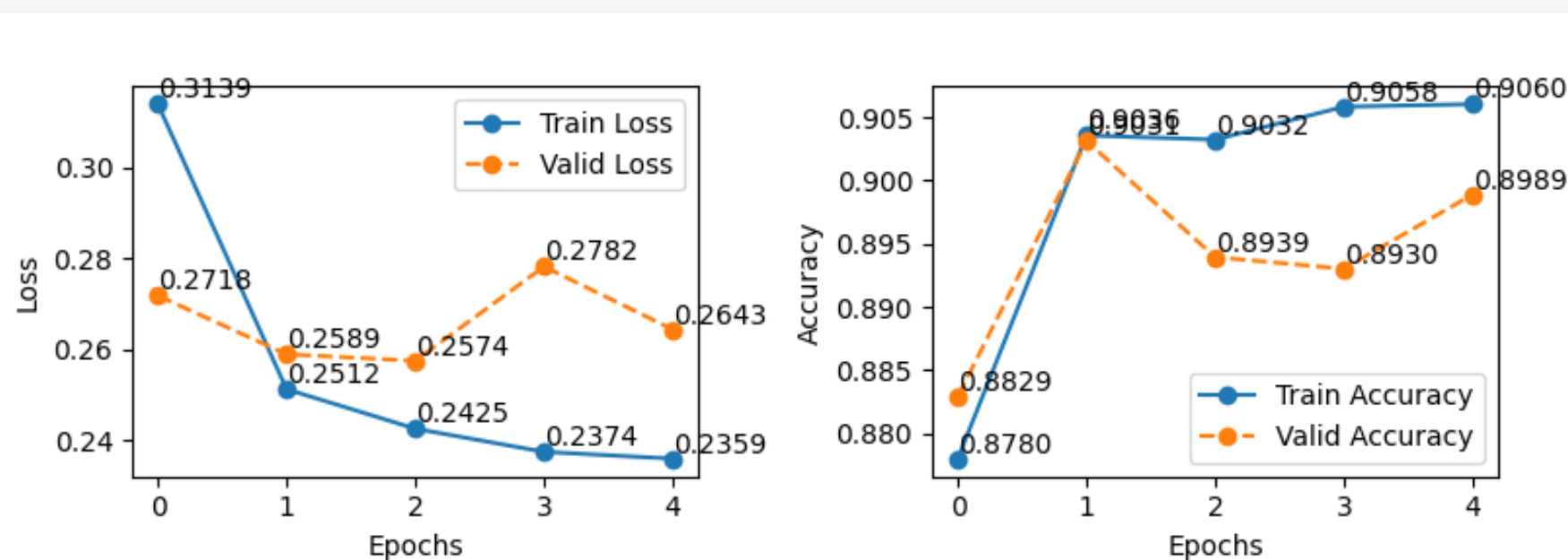
Additional Layers

- **Dropout Layer**, to prevent overfitting
- fully-connected **Linear layer**, which will produce the **logits** for the target word
- **candidate masks** are applied on the logits to **isolate** the only **possible candidates**: the one with the highest score will be the predicted sense.



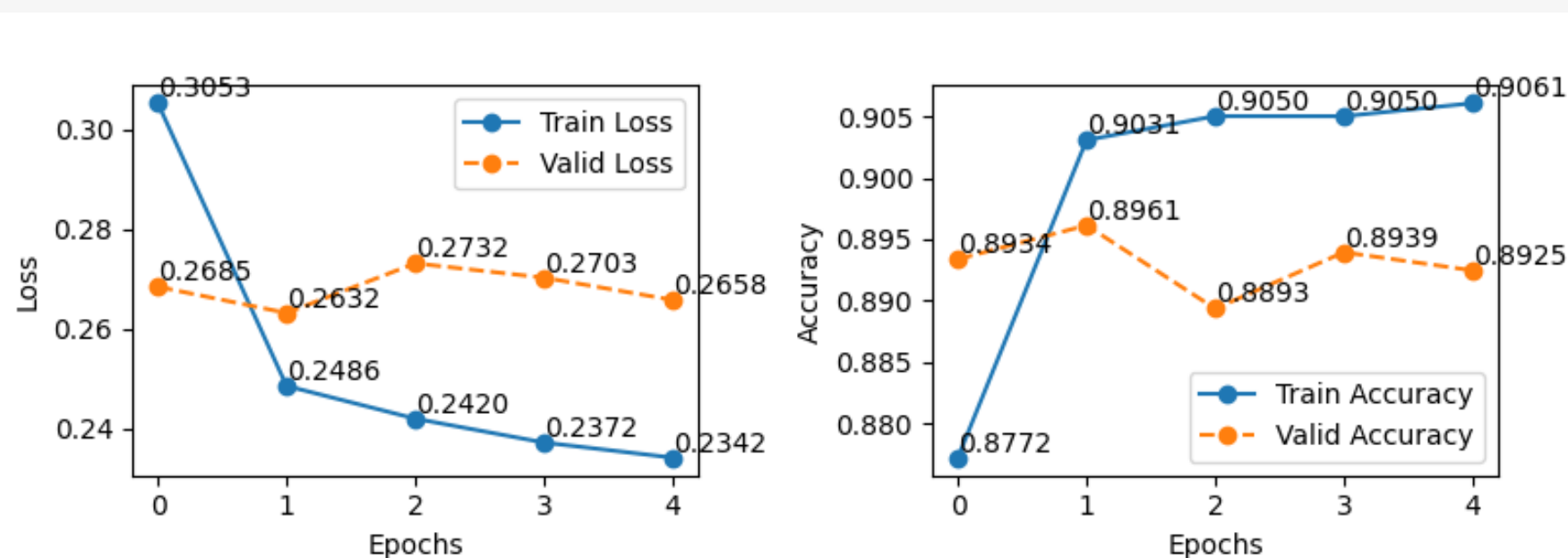
Experiments

Without POS Embeddings



Hyperparameters

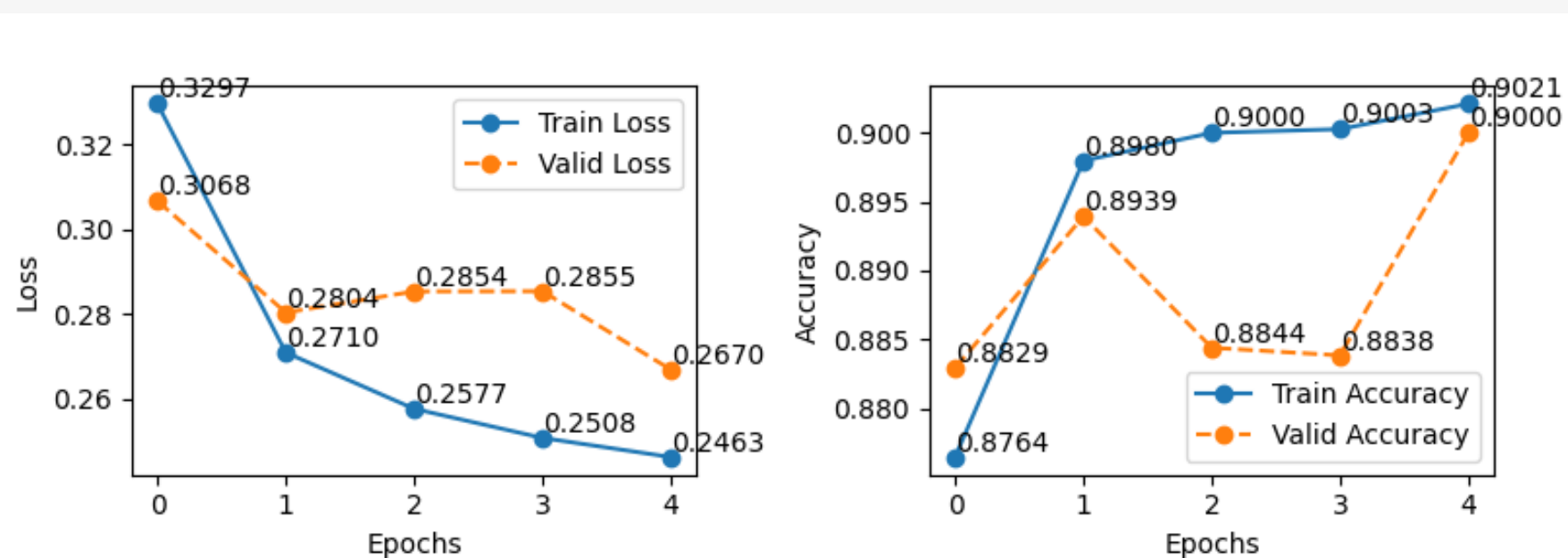
Batch_size: 32
Dropout_rate: 0.1
Learning_rate: 0.001
Model: bert
Num_epochs: 5
Use_lemmas: False
Use_pos: False



Hyperparameters

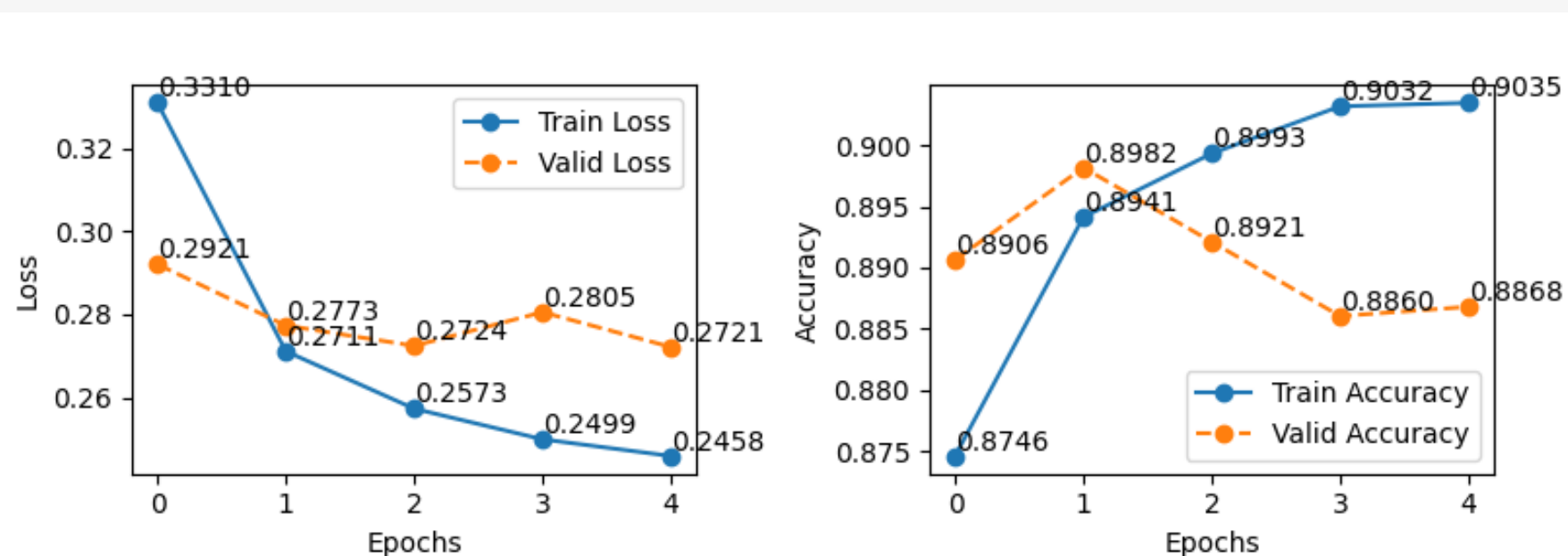
Batch_size: 32
Dropout_rate: 0.1
Learning_rate: 0.001
Model: bert
Num_epochs: 5
Use_lemmas: True
Use_pos: False

With POS Embeddings



Hyperparameters

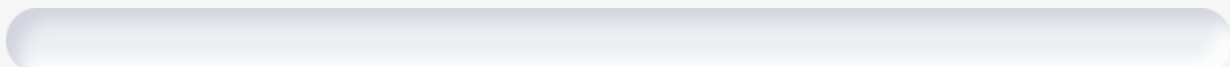
Batch_size: 32
Dropout_rate: 0.1
Learning_rate: 0.001
Model: bert
Num_epochs: 5
Use_lemmas: False
Use_pos: True



Hyperparameters

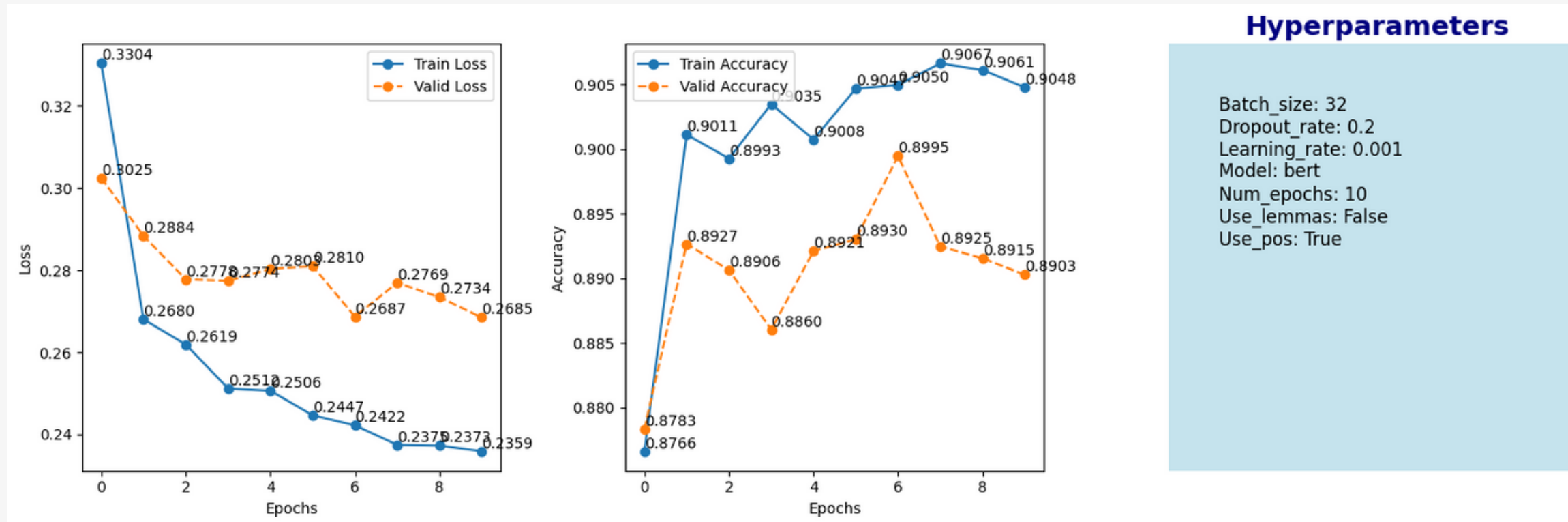
Batch_size: 32
Dropout_rate: 0.1
Learning_rate: 0.001
Model: bert
Num_epochs: 5
Use_lemmas: True
Use_pos: True

Accuracy Results



Experiment	Valid Acc	Test Acc
word_bert	0.9031	0.8893
lemma_bert	0.8961	0.8795
word_pos_bert	0.9000	0.8956
lemma_pos_bert	0.8982	0.8902

With more epochs



Experiment	Valid Acc	Test Acc
word_bert	0.9031	0.8893
lemma_bert	0.8961	0.8795
word_pos_bert	0.9000	0.8956
lemma_pos_bert	0.8982	0.8902
word_pos_bert_10epochs	0.8995	0.8902

Conclusion



- Results showed model with **words as input** win over the **lemma-based** ones
- addition of **POS tags** improved performance
- With **more hyperparameter tuning** and different **combinations of POS embeddings** with transformer input/output, "**word_pos_bert**" model could significantly **improve** in **performance**

**Thank you for the
attention!**